Advanced Data Structures
CS4580

Lecture 05

Kinds Of Complexity

- ✓Worst-case complexity.
- ✓Average complexity.
- Amortized complexity.

Quick Sort

- Sort n distinct numbers.
- Worst-case time is (say) $10n^2$ microseconds on some computer.
- This means that for every n, there is a sequence of n numbers for which quick sort will take $10n^2$ microseconds to complete.
- Also, there is no sequence of n numbers for which quick sort will take more than $10n^2$ microseconds to complete.

Quick Sort

- Average time is (say) $5n \log_2 n$ microseconds on some computer.
- Consider any n, say $n = 1000$.
- Add up the time taken to sort each of the $1000!$ possible 1000 element sequences.
- Divide by $1000!$.
- The result is $5000 \log_2 1000$.

Quick Sort

- What if we sort only 500 of these 1000! sequences?
- We can only conclude that the total time for these 500 sequences will be

  <= 500*(worst-case time)

  = 500*($10n^2$)
- We cannot conclude that the time will be 500*(average time).

Task Sequence

- Suppose that a sequence of n tasks is performed.
- The worst-case cost of a task is c_{wc}.
- Let c_i be the (actual) cost of the ith task in this sequence.
- So, $c_i <= c_{wc}$, $1 <= i <= n$.
- $n * c_{wc}$ is an upper bound on the cost of the sequence.
- $j * c_{wc}$ is an upper bound on the cost of the first j tasks.

Task Sequence

- Let c_{avg} be the average cost of a task in this sequence.
- So, $c_{avg} = \Sigma c_i/n$.
- $n * c_{avg}$ is the cost of the sequence.
- $j * c_{avg}$ is not an upper bound on the cost of the first j tasks.

Task Sequence

- At times, a better upper bound than $j * c_{wc}$ or $n * c_{wc}$ on sequence cost is obtained using amortized complexity.

Amortized Complexity

- The amortized complexity of a task is the amount you charge the task.
- The conventional way to bound the cost of doing a task n times is to use one of the expressions
  - n*(worst-case cost of task)
  - Σ(worst-case cost of task i)
- The amortized complexity way to bound the cost of doing a task n times is to use one of the expressions
  - n*(amortized cost of task)
  - Σ(amortized cost of task i)

Amortized Complexity

- The amortized complexity/cost of individual tasks in any task sequence must satisfy:

  Σ(actual cost of task i)

  <= Σ(amortized cost of task i)

- So, we can use

  Σ(amortized cost of task i)

  as a bound on the actual complexity of the task sequence.

Amortized Complexity

- The amortized complexity of a task may bear no direct relationship to the actual complexity of the task.

Amortized Complexity

- In worst-case complexity analysis, each task is charged an amount that is >= its cost.

  Σ(actual cost of task i)

  <= Σ(worst-case cost of task i)

- In amortized analysis, some tasks may be charged an amount that is < their cost.

  Σ(actual cost of task i)

  <= Σ(amortized cost of task i)

Potential Function

- P(i) = amortizedCost(i) – actualCost(i) + P(i – 1)
- $\Sigma(P(i) - P(i-1)) =$

  $\Sigma(\text{amortizedCost}(i) - \text{actualCost}(i))$
- $P(n) - P(0) = \Sigma(\text{amortizedCost}(i) - \text{actualCost}(i))$
- $P(n) - P(0) >= 0$
- When $P(0) = 0$, $P(i)$ is the amount by which the first i tasks/operations have been over charged.

Arithmetic Statements

- Rewrite an arithmetic statement as a sequence of statements that do not use parentheses.
- a = x+((a+b)*c+d)+y;

  is equivalent to the sequence:

  z1 = a+b;

  z2 = z1*c+d;

  a = x+z2+y;

Arithmetic Statements

a = x+((a+b)*c+d)+y;

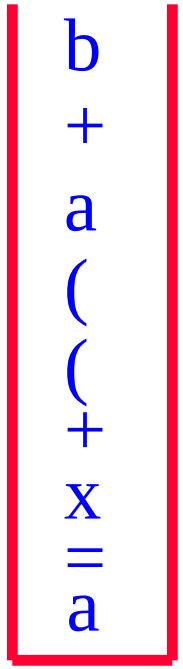
- The rewriting is done using a stack and a method processNextSymbol.
- create an empty stack;

```
for (int i = 1; i <= n; i++)
    // n is number of symbols in statement
    processNextSymbol();
```

Arithmetic Statements

a = x+((a+b)*c+d)+y;

- processNextSymbol extracts the next symbol from the input statement.
- Symbols other than) and ; are simply pushed on to the stack.

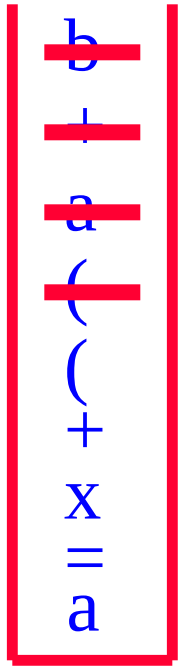


Arithmetic Statements

a = x+((a+b)*c+d)+y;

- If the next symbol is), symbols are popped from the stack up to and including the first (, an assignment statement is generated, and the left hand symbol is added to the stack.

z1 = a+b;

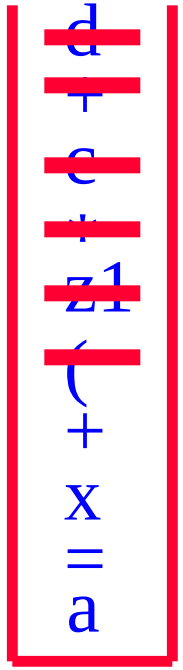


Arithmetic Statements

a = x+((a+b)*c+d)+y;

- If the next symbol is), symbols are popped from the stack up to and including the first (, an assignment statement is generated, and the left hand symbol is added to the stack.

z1 = a+b;
z2 = z1*c+d;

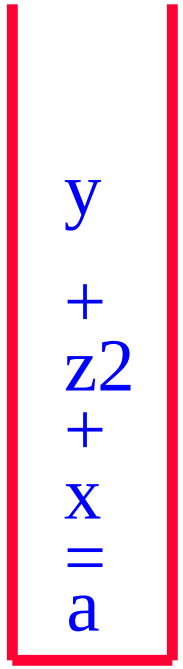


Arithmetic Statements

a = x+((a+b)*c+d)+y;

- If the next symbol is), symbols are popped from the stack up to and including the first (, an assignment statement is generated, and the left hand symbol is added to the stack.

z1 = a+b;
z2 = z1*c+d;

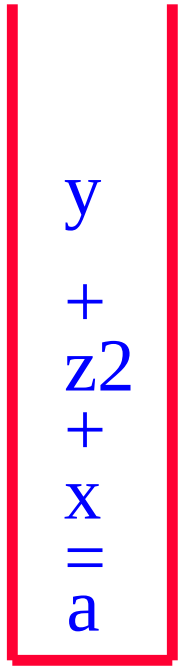


Arithmetic Statements

a = x+((a+b)*c+d)+y;

- If the next symbol is ;, symbols are popped from the stack until the stack becomes empty. The final assignment statement
  a = x+z2+y;
  is generated.

z1 = a+b;
z2 = z1*c+d;



Complexity Of processNextSymbol

a = x+((a+b)*c+d)+y;

- O(number of symbols that get popped from stack)
- O(i), where i is for loop index.

Overall Complexity (Conventional Analysis)

```
create an empty stack;
for (int i = 1; i <= n; i++)
    // n is number of symbols in statement
    processNextSymbol();
```

- So, overall complexity is $O(\Sigma i) = O(n^2)$.
- Although correct, a more careful analysis permits us to conclude that the complexity is $O(n)$.

Ways To Determine Amortized Complexity

- Aggregate method.
- Accounting method.
- Potential function method.

Aggregate Method

- Somehow obtain a good upper bound on the actual cost of the n invocations of processNextSymbol()
- Divide this bound by n to get the amortized cost of one invocation of processNextSymbol()
- Easy to see that

  Σ(actual cost) <= Σ(amortized cost)

Aggregate Method

- The actual cost of the n invocations of processNextSymbol()

  equals number of stack pop and push operations.
- The n invocations cause at most n symbols to be pushed on to the stack.
- This count includes the symbols for new variables, because each new variable is the result of a) being processed. Note that no)s get pushed on to the stack.

Aggregate Method

- The actual cost of the n invocations of processNextSymbol() is at most 2n.
- So, using 2n/n = 2 as the amortized cost of processNextSymbol() is OK, because this cost results in Σ(actual cost) <= Σ(amortized cost)
- Since the amortized cost of processNextSymbol() is 2, the actual cost of all n invocations is at most 2n.

Aggregate Method

- The aggregate method isn't very useful, because to figure out the amortized cost we must first obtain a good bound on the aggregate cost of a sequence of invocations.
- Since our objective was to use amortized complexity to get a better bound on the cost of a sequence of invocations, if we can obtain this better bound through other techniques, we can omit dividing the bound by n to obtain the amortized cost.

Amortized Complexity

- ✓ Aggregate method.
- Accounting method.
- Potential function method.

Potential Function

- P(i) = amortizedCost(i) – actualCost(i) + P(i – 1)
- Σ(P(i) – P(i – 1)) =

  Σ(amortizedCost(i) –actualCost(i))
- P(n) – P(0) = Σ(amortizedCost(i) –actualCost(i))
- P(n) – P(0) >= 0
- When P(0) = 0, P(i) is the amount by which the first i operations have been over charged.

Potential Function Example

	a = x + ((a + b) * c + d) + y ;
actual cost	1 1 1 1 1 11 1 1 5 1 1 1 1 7 1 1 7
amortized cost	2 2 2 2 2 22 2 2 2 2 2 2 2 2 2 2 2
potential	1 2 3 4 5 67 8 9 6 7 8 9 10 5 6 7 2

Potential = stack size except at end.

Accounting Method

- Guess the amortized cost.
- Show that $P(n) - P(0) >= 0$.

Accounting Method Example

```
create an empty stack;
for (int i = 1; i <= n; i++)
    // n is number of symbols in statement
    processNextSymbol();
```

- Guess that amortized complexity of processNextSymbol is 2.
- Start with P(0) = 0.
- Can show that P(i) >= number of elements on stack after ith symbol is processed.

Accounting Method Example

	a = x + ((a + b) * c + d) + y ;
actual cost	1 1 1 1 1 11 1 1 5 1 1 1 1 7 1 1 7
amortized cost	2 2 2 2 2 22 2 2 2 2 2 2 2 2 2 2 2
potential	1 2 3 4 5 67 8 9 6 7 8 9 10 5 6 7 2

- Potential >= number of symbols on stack.
- Therefore, P(i) >= 0 for all i.
- In particular, P(n) >= 0.

Potential Method

- Guess a suitable potential function for which $P(n) - P(0) >= 0$ for all n.
- Derive amortized cost of ith operation using
  $\Delta P = P(i) - P(i-1)$
  = amortized cost – actual cost
- amortized cost = actual cost + ΔP

Potential Method Example

```
create an empty stack;
for (int i = 1; i <= n; i++)
    // n is number of symbols in statement
    processNextSymbol();
```

- Guess that the potential function is P(i) = number of elements on stack after i^{th} symbol is processed (exception is P(n) = 2).
- P(0) = 0 and P(i) – P(0) >= 0 for all i.

i^{th} Symbol Is Not) or ;

- Actual cost of processNextSymbol is 1.
- Number of elements on stack increases by 1.
- $\Delta P = P(i) - P(i-1) = 1$.
- amortized cost = actual cost + ΔP

  $= 1 + 1 = 2$

i^{th} Symbol Is)

- Actual cost of processNextSymbol is #unstacked + 1.
- Number of elements on stack decreases by #unstacked –1.
- $\Delta P = P(i) - P(i-1) = 1 - \#unstacked$.
- amortized cost = actual cost + ΔP

  = #unstacked + 1 +

  (1 – #unstacked)

  = 2

i^{th} Symbol Is ;

- Actual cost of processNextSymbol is #unstacked = P(n–1).
- Number of elements on stack decreases by P(n–1).
- $\Delta P = P(n) - P(n-1) = 2 - P(n-1)$.
- amortized cost = actual cost + ΔP

  $= P(n-1) + (2 - P(n-1))$

  $= 2$

Binary Counter

- n-bit counter
- Cost of incrementing counter is number of bits that change.
- Cost of 001011 => 001100 is 3.
- Counter starts at 0.
- What is the cost of incrementing the counter m times?

Worst-Case Method

- Worst-case cost of an increment is n.
- Cost of 011111 => 100000 is 6.
- So, the cost of m increments is at most mn.

Aggregate Method

0 0 0 0 0

counter

- Each increment changes bit 0 (i.e., the right most bit).
- Exactly floor(m/2) increments change bit 1 (i.e., second bit from right).
- Exactly floor(m/4) increments change bit 2.

Aggregate Method

0 0 0 0 0

counter

- Exactly floor(m/8) increments change bit 3.
- So, the cost of m increments is
  m + floor(m/2) + floor(m/4) + ... < 2m
- Amortized cost of an increment is 2m/m = 2.

Accounting Method

- Guess that the amortized cost of an increment is 2.
- Now show that P(m) – P(0) >= 0 for all m.
- 1st increment:
  - one unit of amortized cost is used to pay for the change in bit 0 from 0 to 1.
  - the other unit remains as a credit on bit 0 and is used later to pay for the time when bit 0 changes from 1 to 0.



2nd Increment.

bits 0 0 0 0 1 → 0 0 0 1 0

credits 0 0 0 0 1 → 0 0 0 1 0

- one unit of amortized cost is used to pay for the change in bit 1 from 0 to 1
- the other unit remains as a credit on bit 1 and is used later to pay for the time when bit 1 changes from 1 to 0
- the change in bit 0 from 1 to 0 is paid for by the credit on bit 0

3rd Increment.

bits	0	0	0	1	0	→	0	0	0	1	1
credits	0	0	0	1	0		0	0	0	1	1

- one unit of amortized cost is used to pay for the change in bit 0 from 0 to 1
- the other unit remains as a credit on bit 0 and is used later to pay for the time when bit 1 changes from 1 to 0

4th Increment.

bits	0 0 0 1 1	→	0 0 1 0 0
credits	0 0 0 1 1		0 0 1 0 0

- one unit of amortized cost is used to pay for the change in bit 2 from 0 to 1
- the other unit remains as a credit on bit 2 and is used later to pay for the time when bit 2 changes from 1 to 0
- the change in bits 0 and 1 from 1 to 0 is paid for by the credits on these bits

Accounting Method

- $P(m) - P(0) = \Sigma(\text{amortizedCost}(i) - \text{actualCost}(i))$
  $= $ amount by which the first m increments have been over charged
  $= $ number of credits
  $= $ number of 1s
  $>= 0$

Potential Method

- Guess a suitable potential function for which $P(n) - P(0) >= 0$ for all n.
- Derive amortized cost of ith operation using
  $\Delta P = P(i) - P(i-1)$
  = amortized cost – actual cost
- amortized cost = actual cost + ΔP

Potential Method

- Guess P(i) = number of 1s in counter after ith increment.
- P(i) >= 0 and P(0) = 0.
- Let q = # of 1s at right end of counter just before ith increment (01001111 => q = 4).
- Actual cost of ith increment is 1+q.
- $\Delta P = P(i) - P(i-1) = 1 - q$ (0100111 => 0101000)
- amortized cost = actual cost + ΔP

$= 1+q + (1 - q) = 2$

Acknowledgements

- E. Horowitz, S. Sahni, and D. Mehta, Fundamentals of data structures in C++, Second Edition, Silicon Press, 2007.
- Lecture Slides of Prof Sartaj Sahni.